Rockchip RT-Thread rkdemo开发指南

文件标识：RK-KF-YF-A36

发布版本：V1.0.0

日期：2024-11-18

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文档主要介绍RT-Thread rkdemo应用的使用方法。

说明

本文档中的所有示例及说明是基于“RK_EVB1_RK3506G_V10_20240511Pana”硬件开发板，以及“rockchip_rk3506_g_evb1_smp_defconfig”软件配置环境进行说明。

产品版本

芯片名称	内核版本
RK3506	RT-Thread 4.1.x

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	郑永智	2024-11-18	初始版本

目录

Rockchip RT-Thread rkdemo开发指南

1. 代码路径
2. 代码编译
   - 2.1 编译命令
   - 2.2 固件位置
   - 2.3 运行结果
3. 模块配置
   - 3.1 SMP配置
   - 3.2 Reset配置
   - 3.3 串口配置
   - 3.4 I2C配置
   - 3.5 PWM配置
   - 3.6 SPI Nand配置
   - 3.7 文件系统配置
   - 3.8 RamFS配置
   - 3.9 USB Device配置
   - 3.10 屏幕及背光配置
   - 3.11 触摸屏配置
   - 3.12 音频模块及声卡配置
   - 3.13 Rockit&RGA配置
   - 3.14 Lvgl配置
   - 3.15 应用配置
4. 开发基础
   - 4.1 Lvgl demo 入口
   - 4.2 显示驱动接口
   - 4.3 输入设备驱动接口
   - 4.4 rkdemo入口函数
   - 4.5 RKdemo主界面入口
   - 4.6 视频播放
     - 4.6.1 视频播放初始化
     - 4.6.2 视频播放参数设置
     - 4.6.3 视频文件播放

1. 代码路径

RT-Thread rkdemo示例的代码路径为applications/lvgl_demo/rk_demo/。

下面是rkdemo相关的主要目录及文件。

```
.
├── lv_app_main.c            # lvgl app demo总入口
├── lv_app_main.h
├── lv_port_indev.c          # lvgl app demo输入设备驱动接口
├── lv_port_indev.h
├── lv_port_disp
│   ├── rkadk
│   │   ├── lv_port_disp.c   # 基于rkadk的lvgl app demo显示设备驱动接口
│   │   └── lv_port_disp.h
│   └── vop
│       ├── lv_port_disp.c   # 基于vop模式的lvgl app demo显示设备驱动接口
│       └── lv_port_disp.h
├── rk_demo                  # rkdemo app目录
│   ├── main.c               # rkdemo app入口
│   ├── main.h
│   ├── home                 # rkdemo主界面
│   ├── furniture_control    # rkdemo家电显控界面
│   ├── intercom_homepage    # rkdemo楼宇对讲界面
│   ├── smart_home           # rkdemo智能家居界面
│   ├── setting              # rkdemo设置菜单界面
│   ├── resource             # rkdemo资源文件目录
│   └── ui_resource.h        # rkdemo资源文件头文件
└── SConscript               # lvgl app demo编译脚本
```

2. 代码编译

SDK发布时，默认已经配置好rkdemo的应用，用户只需要按照下面方法直接编译即可。

2.1 编译命令

```
./build.sh lunch                          # 选择编译工程配置文件
1. rockchip_rk3502_robot_defconfig
2. rockchip_rk3506_b_evb1_defconfig
3. rockchip_rk3506_g_demo_defconfig
4. rockchip_rk3506_g_evb1_amp_defconfig
5. rockchip_rk3506_g_evb1_defconfig
6. rockchip_rk3506_g_evb1_smp_defconfig # 选择RK3506 evb1 smp_defconfig
Which would you like? [1]:
./build.sh                                # 执行工程编译命令
```

2.2 固件位置

编译完成后，固件位于./rockdev/目录下：

```
.
├── amp.img             # AMP固件
├── MiniLoaderAll.bin   # loader
├── parameter.txt       # parameter参数
├── root.img            # rootfs
├── uboot.img           # uboot
└── update.img          # update image
```

2.3 运行结果

固件烧录后，开机显示RT-Thread rkdemo界面。

3. 模块配置

本章节以RK3506 RT-Thread rkdemo应用为例，详细介绍RT-Thread rkdemo应用中涉及的模块配置。参考以下命令打开配置菜单，并参考下面小节进行配置。

```
cd rtos/bsp/rockchip/rk3506-32/          # 进入RT-Thread RK3506 SOC目录
cp board/evb1/smp_defconfig .config      # 将保存的默认配置文件拷贝到.config
scons --menuconfig                       # 执行配置菜单命令，选择配置项，完成后“save”保存
cp .config board/evb1/smp_defconfig      # 保存修改后的配置文件
cd -                                     # 配置完成返回根目录
```

3.1 SMP配置

在RK3506 EVB1 RT-Thread rkdemo中，SMP模式配置如下（其他配置使用默认参数）：

```
RT_USING_SMP [=y]                        # 使能SMP功能
RT_CPUS_NR [=3]                          # CPU个数配置（RK3506支持3个CPU）
```

3.2 Reset配置

该功能支持软件复位。使能该模块，用户可以通过msh命令使系统复位或者进入升级模式。在RK3506 EVB1 RT-Thread rkdemo中配置如下：

```
RT_USING_RESET [=y]
```

msh命令参考如下：

```
msh /> reboot                           # 系统软复位
msh /> reboot loader                    # 进入“loader”升级模式
```

3.3 串口配置

在RK3506 EVB1 RT-Thread rkdemo中，调试串口配置如下（其他配置使用默认参数）：

```
RT_USING_UART [=y]
RT_USING_UART0 [=y]                     # 使能串口0（RK3506 EVB1使用串口0作为LOG输出）
```

3.4 I2C配置

在RK3506 EVB1 RT-Thread rkdemo中，Acodec和TP模块需要使用到I2C设备，配置如下（其他配置使用默认参数）：

```
RT_USING_I2C [=y]                       # 使能I2C驱动
RT_USING_I2C0 [=y]                      # ES8388依赖于I2C0
RT_USING_I2C2 [=y]                      # TP接口依赖于I2C2
```

3.5 PWM配置

在RK3506 EVB1 RT-Thread rkdemo中，屏幕背光需要使用PWM设备，配置如下（其他配置使用默认参数）：

```
RT_USING_PWM [=y]                       # 使能PWM功能
RT_USING_PWM0 [=y]                      # 背光使用PWM0
```

3.6 SPI Nand配置

在RK3506 EVB1 RT-Thread rkdemo中，SPI Nand Flash配置如下（其他配置使用默认参数）：

```
RT_USING_MTD_NAND [=y]                  # 使能nand flash驱动
RT_USING_SPINAND [=y]                   # 使能spi nand flash
RT_USING_SPINAND_FSPI_HOST [=y]         # 使能FSPI接口
RT_USING_FTL [=y]                       # 使能FTL算法
RT_USING_DHARA [=y]                     # 使能DHARA算法
RT_USING_DHARA_ENABLE_CACHE [=y]        # 使能DHARA CACHE
```

3.7 文件系统配置

在RK3506 EVB1 RT-Thread rkdemo中，文件系统配置如下（其他配置使用默认参数）：

```
RT_USING_DFS [=y]                       # 使能文件系统
RT_USING_DFS_MNTTABLE [=y]              # 使能mount table挂载文件系统
RT_USING_DFS_ELMFAT [=y]                # 使能elm文件系统
RT_DFS_ELM_MAX_SECTOR_SIZE [=2048]      # 配置扇区大小（SpiNand为2048
DFS_FD_MAX [=1024]                      # 最大支持文件个数（rkdemo需要配置1024）
```

3.8 RamFS配置

Rams是在内存中分配一片区域，实现文件系统接口功能。用于方便用户调试，以及提升代码运行效率等。

在RK3506 EVB1 RT-Thread rkdemo中，RamFs用于存储UI资源文件，以提升UI显示效果。配置如下：

```
RT_USING_DFS_RAMFS [=y]                 # 使能RamFS
RT_DFS_RAMFS_SIZE [=10485760]           # RamFs大小配置（rkdemo中配置为10MB）
```

在RK3506 EVB1 RT-Thread rkdemo中，RamFs的使用如下：

```
/*rtos/applications/lvgl_demo/rk_demo/ui_resource.h*/
#ifdef RT_USING_DFS_RAMFS
#define PATH_PREFIX     "/ramfs"     // UI资源路径修改为RamFs
#else
#define PATH_PREFIX     "/resource"  // UI资源原始路径
#endif

/*rtos/applications/lvgl_demo/rk_demo/main.c*/
void lv_rk_demo(void)
{
#ifdef RT_USING_DFS_RAMFS
    copy("resource/", "/ramfs");     // 将资源文件从Flash的"resource"目录中，拷贝到RamFs根目录
#endif
    //......
}
```

在该示例中，rkdemo初始化时，先将原先存储在SpiNand flash "resource"目录下的资源文件，拷贝到Ramfs根目录下。在系统使用时，从Ramfs中读取相关资源文件，提高显示运行效率。

3.9 USB Device配置

USB device用于将本地磁盘通过USB连接线连接电脑，拷贝用户数据。在RK3506 EVB1 RT-Thread rkdemo中，将"userdata"分区划分为用户分区。用户可以通过USB device功能将文件拷贝到“userdata”分区。USB device配置如下（其他配置使用默认参数）：

```
RT_USING_USB_DEVICE [=y]                # 使能USB device
_RT_USB_DEVICE_MSTORAGE [=y]            # 选择Mass Storage功能
RT_USB_MSTORAGE_DISK_NAME [=userdata]   # USB device挂载分区名称
```

3.10 屏幕及背光配置

在RK3506 EVB1 RT-Thread rkdemo中，屏幕及背光配置如下（其他配置使用默认参数）：

```
RT_USING_DISPLAY [=y]                   # 使能显示驱动
RT_USING_VOP [=y]                       # 使能VOP驱动
RT_USING_DW_MIPI_DSI [=y]               # 选择MIPI接口
RT_USING_INNO_MIPI_DPHY [=y]            # 使能MIPU DPHY
RT_USING_PANEL_ILI9881D [=y]            # 选择MIPI屏幕（RK3506 EVB1为ILI9881D）
RT_USING_BACKLIGHT [=y]                 # 使能背光功能
```

3.11 触摸屏配置

在RK3506 EVB1 RT-Thread rkdemo中，触摸屏配置如下（其他配置使用默认参数）：

```
RT_USING_TOUCH [=y]                     # 使能TP驱动
RT_TOUCH_PIN_IRQ [=y]                   # 使能TP GPIO中断
RT_USING_TOUCH_DRIVERS [=y]             # 使能Rockchip TP驱动
RT_USING_TOUCH_GT1X [=y]                # RK3506 EVB1使用GT1x驱动
RT_TOUCH_X_RANGE [=720]                 # 配置TP屏幕宽高
RT_TOUCH_Y_RANGE [=1280]                # 配置TP屏幕宽高
```

3.12 音频模块及声卡配置

在RK3506 EVB1 RT-Thread rkdemo中，音频及声卡配置如下（其他配置使用默认参数）：

```
RT_USING_AUDIO [=y]                     # 使能Audio
RT_USING_AUDIO_CARD [=y]                # 使能声卡
RT_USING_AUDIO_CARD_ES8388 [=y]         # 选择声卡（RK3506 EVB1为ES8388）
```

3.13 Rockit&RGA配置

在RK3506 EVB1 RT-Thread rkdemo中，Rockit&RGA配置如下（其他配置使用默认参数）：

```
RT_USING_RGA [=y]                       # 使能RGA
RT_USING_MULTIMEDIA [=y]                # 使能Rockit&RKADK
RT_USING_ROCKIT [=y]                    # 使能Rockit
RT_USING_RKADK [=y]                     # 使能RKADK
RT_RKADK_ENABLE_DISPLAY [=y]            # 使能RKADK显示模块，其他使用默认配置

RT_USING_LWIP [=y]                      # 使能LwIP v2.1.2，Rockit中使用部分lwip中的函
数
RT_USING_LWIP212 [=y]                   # 使能LwIP v2.1.2，Rockit中使用部分lwip中的函
数
```

3.14 Lvgl配置

在RK3506 EVB1 RT-Thread rkdemo中，Lvgl配置如下（其他配置使用默认参数）：

```
RT_USING_LVGL [=y]                      # 使能lvgl
RT_LV_COLOR_DEPTH_32 [=y]               # 位宽32bit
RT_LV_COLOR_SCREEN_TRANSP [=y]          # 支持透明色
LV_DISP_DEF_REFR_PERIOD [=16]           # 屏幕刷新周期60fps
LV_INDEV_DEF_READ_PERIOD [=16]          # 触摸屏采集周期60fps
LV_IMG_CACHE_DEF_SIZE [=500]            # 使能Cache
RT_LV_USE_FS_POSIX [=y]                 # 使能文件系统接口
LV_FS_POSIX_LETTER [=65]                # 文件前缀“A”
RT_LV_USE_PNG [=y]                      # 使能PNG解码
RT_LV_USE_SJPG [=y]                     # 使能SJPG解码
RT_LV_USE_FREETYPE [=y]                 # 使能Freetype字体接口
RT_USING_FREETYPE [=y]                  # 使能Freetype
```

3.15 应用配置

在RK3506 EVB1 RT-Thread中，rkdemo应用配置如下（其他配置使用默认参数）：

```
RT_USING_LVGL_DEMO [=y]                 # 使能app lvgl demo应用
RT_LV_RK_DEMO [=y]                      # 使能app rkdemo
```

在RK3506 EVB1 RT-Thread中，lvgl music应用配置如下（其他配置使用默认参数）：

```
RT_LVGL_USING_DEMOS [=y]                # 使能lvgl内置demo
RT_LV_USE_DEMO_MUSIC [=y]               # 使能lvgl内置music demo
RT_USING_LVGL_DEMO [=y]                 # 使能app lvgl demo应用
RT_LV_DEMO_MUSIC [=y]                   # 使能app lvgl music demo调用
```

4. 开发基础

4.1 Lvgl demo 入口

lvgl demo app入口函数位于：rtos/applications/lvgl_demo/lv_app_main.c。

主要相关说明如下：

```
static void lvgl_task(void *arg)
{
    //......

    lv_init();                              // lvgl初始化

#ifdef RT_USING_RKADK
    lv_port_disp_init(LV_DISP_ROT_90);    // 显示屏旋转90度
    lv_port_indev_init(LV_DISP_ROT_90);   // 触摸屏旋转90度
#else
    lv_port_disp_init(LV_DISP_ROT_NONE);  // 屏幕不旋转
    lv_port_indev_init(LV_DISP_ROT_NONE); // 屏幕不旋转
#endif

    //......

#ifdef RT_LV_DEMO_MUSIC
    //......
#elif defined(RT_LV_RK_DEMO)
    lv_rk_demo();                           // rkdemo初始化
#endif

    while (1)
    {
        lv_task_handler();
        rt_thread_mdelay(2);
    }
}

int lvgl_app_init(void)
{
    rt_thread_t lvgl;

    lvgl = rt_thread_create("lvgl", lvgl_task, RT_NULL, 64*1024, 4, 20);
    rt_thread_startup(lvgl);

    return RT_EOK;
}

INIT_APP_EXPORT(lvgl_app_init);           // 上电后自动运行app demo
```

函数说明如下：

函数	说明
lvgl_app_init()	lvgl demo总入口，创建lvgl demo task。 根据需求修改堆栈大小。在该示例中，堆栈要求64KB或以上。
lvgl_task()	lvgl demo task。初始化lvgl，并且根据配置运行相应的app demo。 RKADK模式支持屏幕旋转，VOP模式不支持屏幕旋转。

4.2 显示驱动接口

目前lvgl demo支持两种显示驱动接口，区别如下：

操作	VOP模式	RKADK模式
代码路径	lv_port_disp/vop/	lv_port_disp/rkadk/
送显方式	直接调用屏幕显示驱动送显。	通过调用RKADK接口，将显示数据送给RKADK，由RKADK合成处理后送显。
旋转	不支持屏幕旋转	支持屏幕旋转

由于两种实现方式功能基本一致，本文将以RKADK送显模式进行主要函数接口说明。

显示驱动初始化函数接口。列出主要接口调用及说明如下：

```
void rk_disp_flush(lv_disp_drv_t * disp_drv, const lv_area_t * area, lv_color_t *
color_p)
{
    MB_BLK blk;

    //将绘制显示区域数据拷贝到显示buffer
    for(y = area->y1; y <= area->y2; y++) {
        lv_color_t *disp = disp_fb + y * disp_drv->hor_res + area->x1;
        memcpy(disp, color_p, w * sizeof(lv_color_t));
        color_p += w;
    }

    //从RKADK显示buffer队列中获取一帧显示buffer
    blk = RK_MPI_MB_GetMB(disp_dev.buf_pool, disp_dev.draw_buf.size, RK_TRUE);
    dst_ptr = RK_MPI_MMZ_Handle2VirAddr(blk);

    //调用RGA接口将显示数据送到RKADK buffer
    src_img = wrapbuffer_physicaladdr(disp_fb, disp_drv->hor_res,
                                      disp_drv->ver_res, format);
    dst_img = wrapbuffer_physicaladdr(dst_ptr, disp_drv->hor_res,
                                      disp_drv->ver_res, format);
    ret = imcheck(src_img, dst_img, (im_rect){}, (im_rect){});
    ret = imcopy(src_img, dst_img);

    //调用RKADK接口将数据送到显示屏幕
    RK_MPI_SYS_MmzFlushCache(blk, RK_FALSE);
    RKADK_UI_Update(disp_dev.ui_ptr, &(disp_dev.frame_info));
```

```
    RK_MPI_MB_ReleaseMB(blk);

    //调用lvgl接口刷新
    lv_disp_flush_ready(disp_drv);
}

rt_err_t lv_port_disp_init(lv_disp_rot_t rotate_disp)
{
    // 显示驱动初始化
    int ret = rk_disp_init(rotate_disp);

    // 获取屏幕硬件宽、高参数
    rk_disp_get_sizes(&lcd_w, &lcd_h, NULL);

    // 调用RKADK接口，申请缓冲buffer
    ret = RK_MPI_MMZ_Alloc(&blk, size_in_px_cnt * (LV_COLOR_SIZE >> 3),
                           RK_MMZ_ALLOC_CACHEABLE);
    disp_dev.draw_buf.map = RK_MPI_MMZ_Handle2VirAddr(blk);
    disp_dev.draw_buf.fd = RK_MPI_MMZ_Handle2Fd(blk);

    //lvgl绘制buffer初始化
    lv_disp_draw_buf_init(&draw_buf_dsc, disp_dev.draw_buf.map, NULL,
                          size_in_px_cnt);

    //lvgl显示驱动初始化
    lv_disp_drv_init(&disp_drv);

    //根据屏幕旋转参数，设置显示驱动宽、高参数
    if (rotate_disp == LV_DISP_ROT_NONE)
    {
        disp_drv.hor_res = lcd_w;
        disp_drv.ver_res = lcd_h;
    }
    else if (rotate_disp == LV_DISP_ROT_90)
    {
        disp_drv.hor_res = lcd_h;
        disp_drv.ver_res = lcd_w;
    }
    else if (rotate_disp == LV_DISP_ROT_270)
    {
        disp_drv.hor_res = lcd_h;
        disp_drv.ver_res = lcd_w;
    }
    //......

    //注册lvgl显示操作相关回调函数接口
    disp_drv.flush_cb = rk_disp_flush;
    disp_drv.rounder_cb = rk_disp_rounder;
    disp_drv.draw_buf = &draw_buf_dsc;

    //注册lvgl显示驱动
    lv_disp_drv_register(&disp_drv);

    return RT_EOK;
}
```

4.3 输入设备驱动接口

lvgl 输入设备驱动接口文件为：rtos/applications/lvgl_demo/lv_port_indev.c。

主要接口函数说明如下：

```
static void lv_port_touch_block_init(struct rt_touchpanel_block *block)
{
    //根据旋转参数设置触摸屏宽、高参数
    if ((rot == LV_DISP_ROT_90) ||
        (rot == LV_DISP_ROT_270))
    {
        block->w = LVGL_FB_HEIGHT;
        block->h = LVGL_FB_WIDTH;
    }
    else
    {
        block->w = LVGL_FB_WIDTH;
        block->h = LVGL_FB_HEIGHT;
    }
}

void lv_port_indev_init(lv_disp_rot_t rotate)
{
    //lvgl显示驱动初始化
    lv_indev_drv_init(&indev_drv);

    //注册lvgl显示驱动
    indev_drv.type = LV_INDEV_TYPE_POINTER;
    indev_drv.read_cb = touchpad_read;
    lv_indev_drv_register(&indev_drv);
}
```

4.4 rkdemo入口函数

RT-Thread rkdemo入口文件为：rtos/applications/lvgl_demo/rk_demo/main.c

入口函数如下：

```
void lv_rk_demo(void)
{
    // 初始化时，将资源文件从Flash文件系统中拷贝到RAMFS，以提高执行效率
#ifdef RT_USING_DFS_RAMFS
    copy("/usr/share/resource/", "/ramfs");
#endif

    // UI、字库等相关初始化
    lvgl_init();
    app_init();

    // 调用主界面初始化函数
    rk_demo_init();
```

```
}
```

4.5 RKdemo主界面入口

RT-Thread rkdemo主界面入口文件为：rtos/applications/lvgl_demo/rk_demo/home/home_ui.c

```
void rk_demo_init(void)
{
    //该函数主要进行lvgl UI画布背景初始化，以及调用主界面UI lvgl绘制操作
    //具体参考实现代码
}
```

4.6 视频播放

RT-Thread rkdemo视频播放接口文件为：
rtos/applications/lvgl_demo/rk_demo/furniture_control/player_ui.c

视频操作相关接口为：

函数名称	说明
rkadk_init()	RKADK视频播放初始化，用于配置视频操作相关参数，并创建播放器
rkadk_deinit()	RKADK视频播放反初始化
param_init()	RKADK视频播放参数设置
PlayerEventFnTest()	RKADK视频播放事件回调函数

4.6.1 视频播放初始化

```
static void rkadk_init(void)
{
    //RKADK视频播放相关初始化处理
    setenv("rt_vo_disable_vop", "0", 1);
    RKADK_MPI_SYS_Init();
    RKADK_PARAM_Init(NULL, NULL);

    //视频播放参数初始化
    memset(&stPlayCfg, 0, sizeof(RKADK_PLAYER_CFG_S));
    param_init(&stPlayCfg.stFrmInfo);

    //声卡配置，根据不同硬件资源指定声卡名称
    stPlayCfg.stAudioCfg.pSoundCard = "es8388p";     //声卡名称
    stPlayCfg.stAudioCfg.u32SpeakerVolume = 70;

    stPlayCfg.bEnableVideo = 1;                      //使能视频
    stPlayCfg.bEnableAudio = 1;                      //使能音频
    stPlayCfg.stFrmInfo.u32FrmInfoX = RK_PCT_H(33); //视频显示区域宽高设置
```

```
    stPlayCfg.stFrmInfo.u32FrmInfoY = RK_PCT_W(2);  //视频显示区域宽高设置
    //stPlayCfg.bEnableBlackBackground = true;      //视频播放完成后是否输出黑屏
    stPlayCfg.pfnPlayerCallback = PlayerEventFnTest;//事件回调函数
    stPlayCfg.stVdecCfg.u32FrameBufCnt = 4;

    //调用RKADK接口创建视频播放器
    if (RKADK_PLAYER_Create(&pPlayer, &stPlayCfg))
    {
        printf("rkadk: RKADK_PLAYER_Create failed\n");
        return;
    }
}
```

4.6.2 视频播放参数设置

```
static void param_init(RKADK_PLAYER_FRAME_INFO_S *pstFrmInfo)
{
    // 参数检查
    RKADK_CHECK_POINTER_N(pstFrmInfo);

    memset(pstFrmInfo, 0, sizeof(RKADK_PLAYER_FRAME_INFO_S));

    // 设置视频显示宽、高参数
    pstFrmInfo->u32DispWidth = RK_PCT_H(57);
    pstFrmInfo->u32DispHeight = RK_PCT_W(96);
    pstFrmInfo->u32ImgWidth = pstFrmInfo->u32DispWidth;
    pstFrmInfo->u32ImgHeight = pstFrmInfo->u32DispHeight;

    pstFrmInfo->u32VoLay = 0;  // rkadk select the default first device
    pstFrmInfo->u32VoChn = 1;  // ui is 2 . play is 1
    pstFrmInfo->u32VoDev = -1; // rkadk select the default first device
    pstFrmInfo->enIntfSync = RKADK_VO_OUTPUT_DEFAULT;
    pstFrmInfo->enVoSpliceMode = SPLICE_MODE_RGA; // rkadk depend chips
    pstFrmInfo->u32EnIntfType = DISPLAY_TYPE_MIPI;

    pstFrmInfo->u32BorderColor = 0x0000FA;
    pstFrmInfo->bMirror = RKADK_FALSE;
    pstFrmInfo->bFlip = RKADK_FALSE;
    //旋转参数：0不旋转，1 90度，2 180度，3 270度
    pstFrmInfo->u32Rotation = 1;

    //其余参数使用默认配置
    pstFrmInfo->stSyncInfo.bIdv = RKADK_TRUE;
    pstFrmInfo->stSyncInfo.bIhs = RKADK_TRUE;
    pstFrmInfo->stSyncInfo.bIvs = RKADK_TRUE;
    pstFrmInfo->stSyncInfo.bSynm = RKADK_TRUE;
    pstFrmInfo->stSyncInfo.bIop = RKADK_TRUE;
    pstFrmInfo->stSyncInfo.u16FrameRate = 30;
    pstFrmInfo->stSyncInfo.u16PixClock = 65000;
    pstFrmInfo->stSyncInfo.u16Hact = 1200;
    pstFrmInfo->stSyncInfo.u16Hbb = 24;
    pstFrmInfo->stSyncInfo.u16Hfb = 240;
    pstFrmInfo->stSyncInfo.u16Hpw = 136;
```

```
    pstFrmInfo->stSyncInfo.u16Hmid = 0;
    pstFrmInfo->stSyncInfo.u16Vact = 1200;
    pstFrmInfo->stSyncInfo.u16Vbb = 200;
    pstFrmInfo->stSyncInfo.u16Vfb = 194;
    pstFrmInfo->stSyncInfo.u16Vpw = 6;

    return;
}
```

4.6.3 视频文件播放

SDK默认指定视频文件路径为用户磁盘根目录：data/

用户在播放视频文件之前，首先需要打开USB device配置，将视频文件拷贝到用户盘根目录。

用户如果需要修改磁盘路径，可以在rtos/applications/lvgl_demo/rk_demo/furniture_control/player_ui.c中修改视频文件播放路径，参考如下：

```
#define MAX_FILE_COUNT 10       // 最大支持文件个数
#define PATH_VIDEO "/data/"     // 视频文件路径，默认为用户磁盘根目录
```